
matplotlib

Matplotlib

Matplotlib is a low level graph plotting library in python that serves as a visualization utility.

Matplotlib

Created By-	John D. Hunter
Developed-	2002
Release-	2003

How to install Matplotlib in Google Colab

- Matplotlib is generally pre-installed in Google Colab
- To install or update Matplotlib, run:

  - !pip install matplotlib

How to use Matplotlib

```
import matplotlib as mt
```

Matplotlib Pyplot

Most of the Matplotlib utilities lies under the pyplot submodule, and are usually imported under the plt alias:

```
import matplotlib.pyplot as plt
```

Creating line chart

- A line chart or line graph is a type of chart which displays information as a series of data points called 'marker, connected by straight line segments.
- The PyPlot interface offers plot() function for creating a line graph.

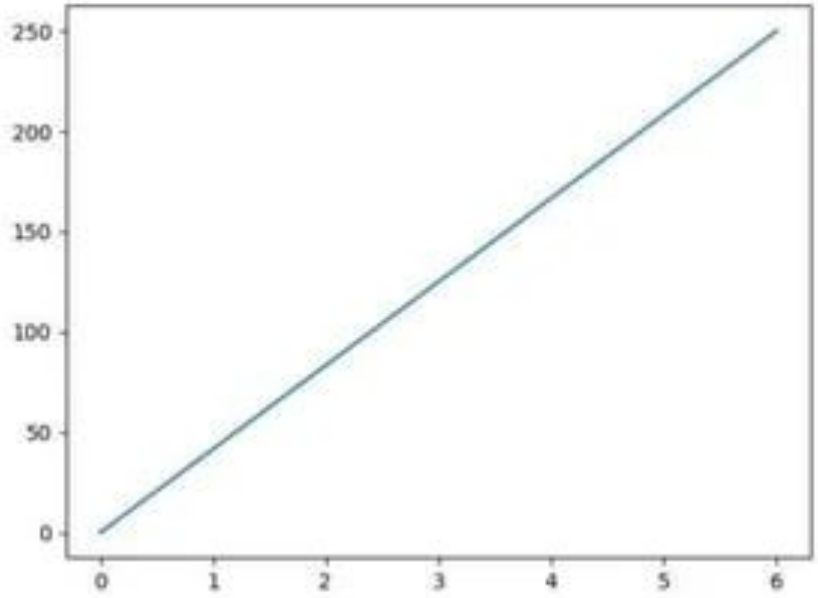
Plotting x and y points

- The plot() function is used to draw points (markers) in a diagram.
- By default, the plot() function draws a line from point to point.
- The function takes parameters for specifying points in the diagram.
- Parameter 1 is an array containing the points on the x-axis (horizontal axis).
- Parameter 2 is an array containing the points on the y-axis (vertical axis)..

Plotting Line

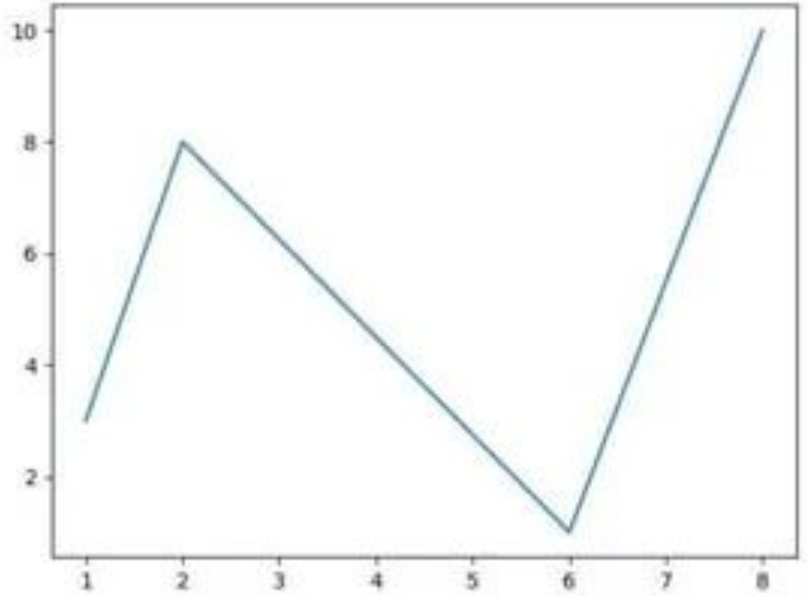
Draw a line in a diagram from position (0,0) to position (6,250):

```
import matplotlib.pyplot as plt
import numpy as np

x= np.array([0, 6])
y = np.array([0, 250])

plt.plot(x, y)
plt.show()
```



Multiple Points

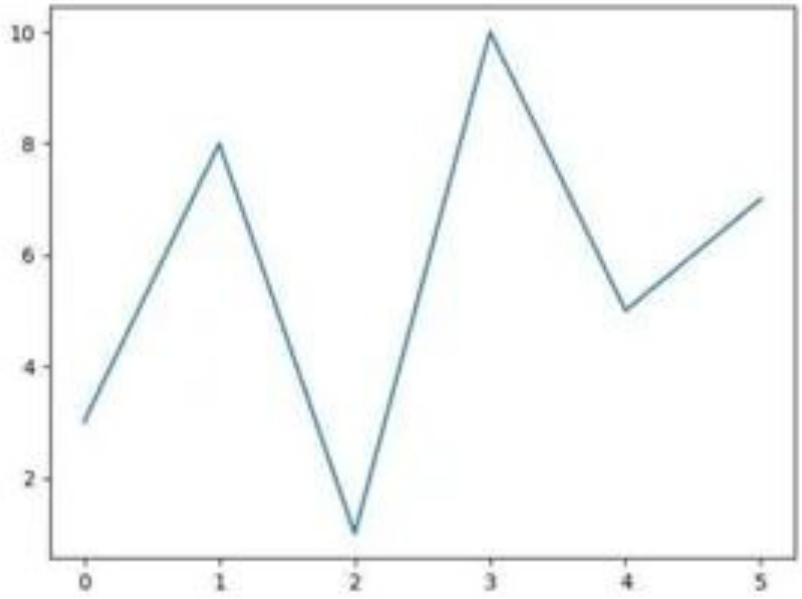
```
import matplotlib.pyplot as plt
import numpy as np

x= np.array([1, 2,6,8])
y = np.array([3,8,1,10])

plt.plot(x, y)
plt.show()
```



Default X-Axis

If we do not specify the points in the x-axis, they will get the default values 0, 1, 2, 3, (etc. depending on the length of the y-points.

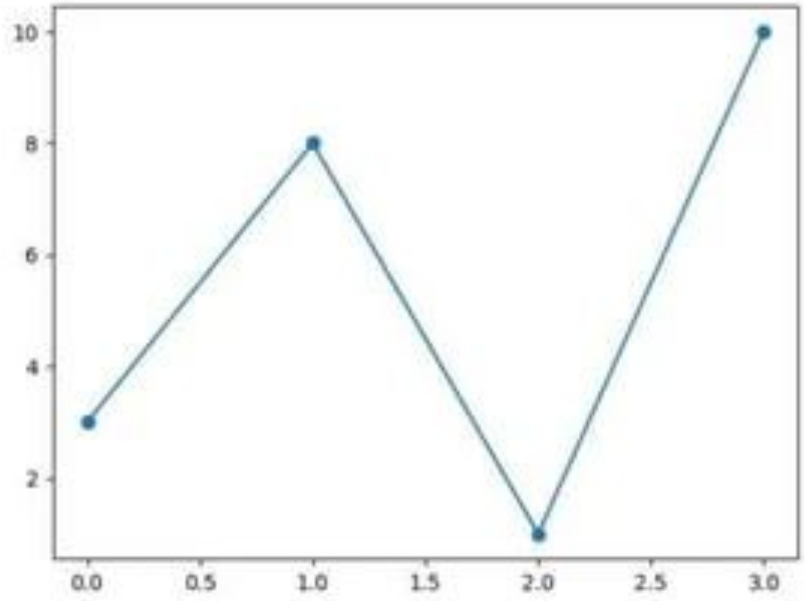
```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([3,8,1,10,5,7])

plt.plot( y)
plt.show()
```



Matplotlib Markers

You can use the keyword argument marker to emphasize each point with a specified marker:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([3,8,1,10])

plt.plot( y, marker='o')
plt.show()
```



Markers Reference

Marker	Description
'o'	Circle
'*'	Star
'.'	Point
'x'	X
'X'	X (filled)
'+'	Plus
'P'	Plus (filled)
's'	Square
'D'	Diamond

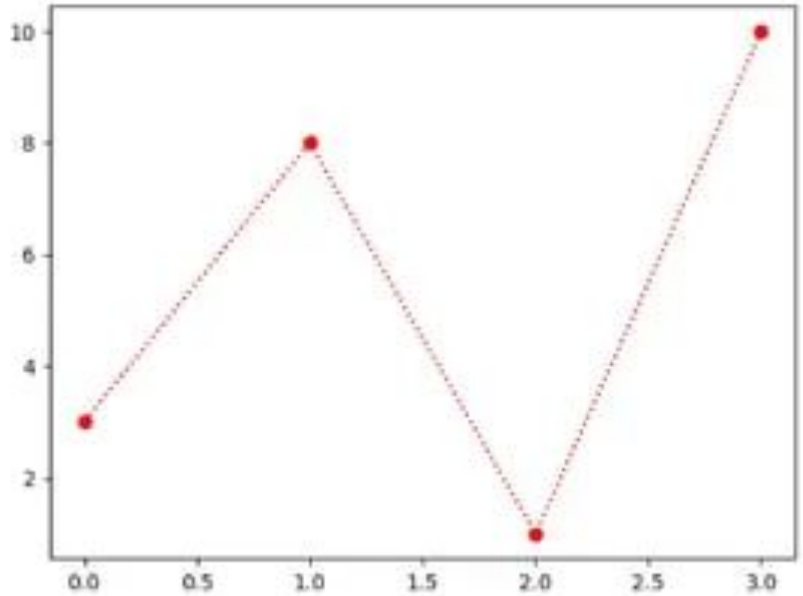
Marker	Description
'd'	Diamond (thin)
'p'	Pentagon
'H'	Hexagon
'h'	Hexagon
'v'	Triangle Down
'^'	Triangle Up
'<'	Triangle Left
'>'	Triangle Right

Format Strings fmt

This parameter is also called fmt, and is written with this syntax:

marker|line|color

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([3,8,1,10])

plt.plot( y,  'o:r')
plt.show()
```



Line Reference

Line Syntax	Description
'-'	Solid Line
':'	Dotted Line
'--'	Dashed Line
'-.'	Dashed/ Dotted Line

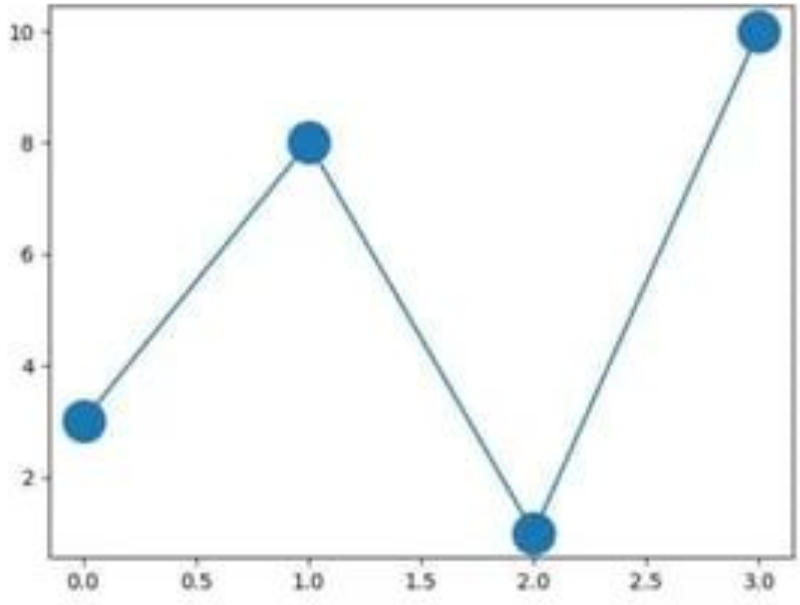
Color Reference

Color Syntax	Description
'r'	Red
'g'	Green
'b'	Blue
'c'	Cyan
'm'	Magenta
'y'	Yellow
'k'	Black
'w'	White

Markers Size

You can use the keyword argument markersize or the shorter version, ms to set the size of the markers:

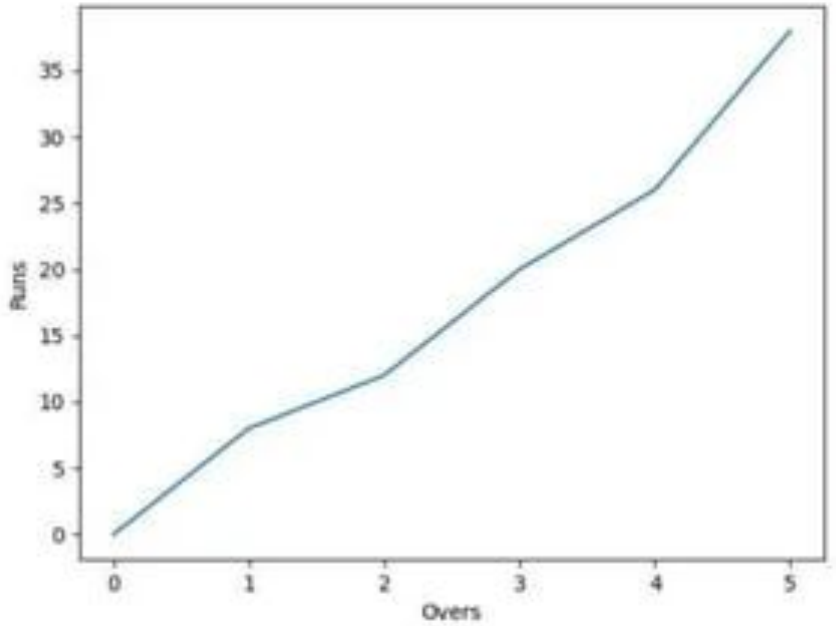
```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([3,8,1,10])

plt.plot( y,  marker= 'o', ms= 20)
plt.show()
```



Create Labels

With Pyplot, you can use the xlabel() and ylabel() functions to set a label for the x- and y-axis.

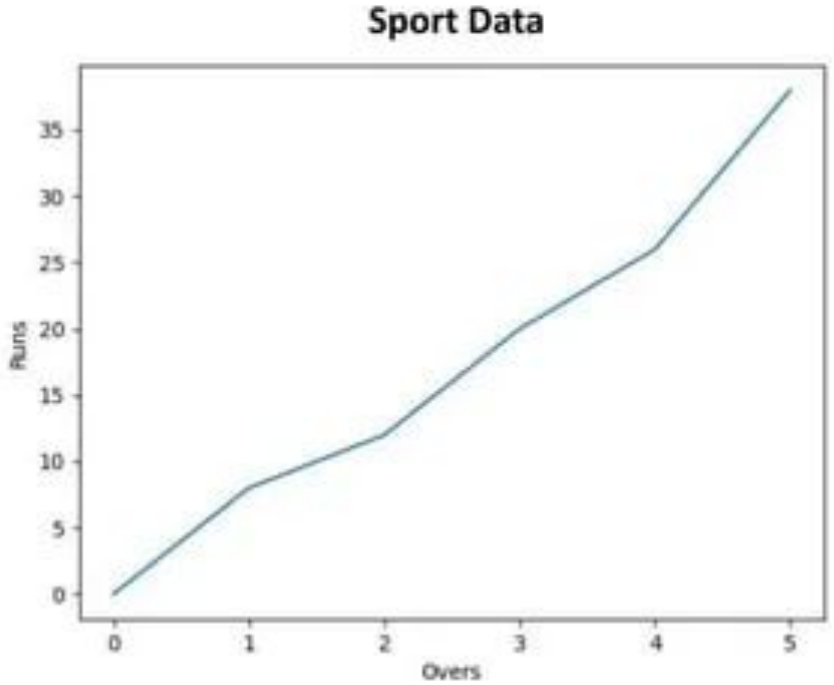
```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([0,1,2,3,4,5])
y = np.array([0,8,12,20,26,38])

plt.plot(x,y)
plt.xlabel("Overs')
plt.ylabel("Runs')
plt.show()
```



Create Title

With Pyplot, you can use the title() function to set a title for the plot.

```
 matplotlib.pyplot as plt
import numpy as np
x = np.array([0,1,2,3,4,5])
y = np.array([0,8,12,20,26,38])
plt.plot(x,y)
plt.xlabel("Overs')
plt.ylabel("Runs')
Plt.title("Sport Data")
plt.show()
```

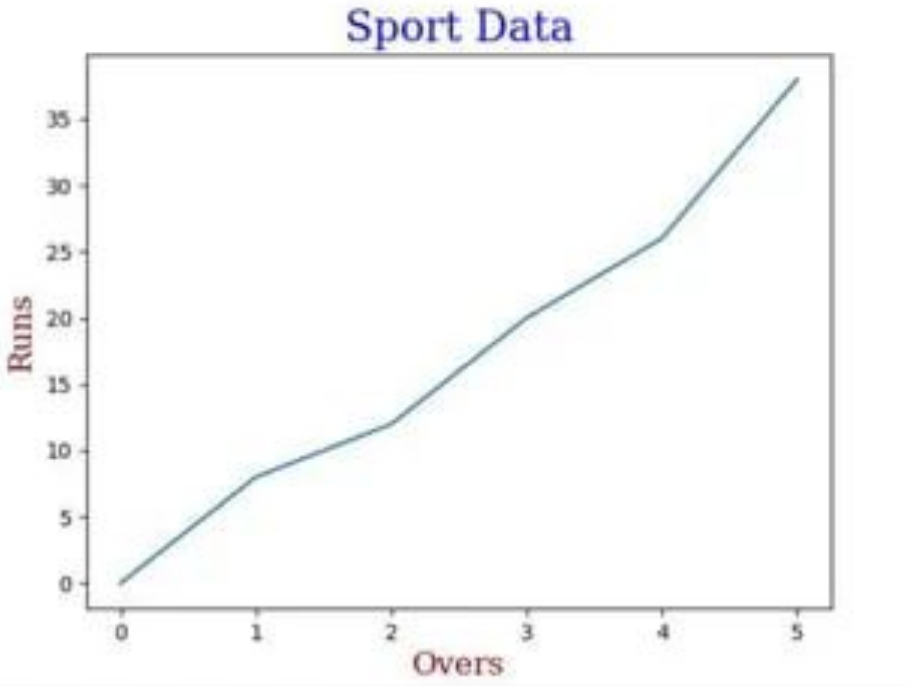


Set Font Properties for Title and Labels

You can use the fontdict parameter in xlabel(), ylabel(), and title() to set font properties for the title and labels.

```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([0,1,2,3,4,5])
y = np.array([0,8,12,20,26,38])
font1 = {'family':'serif','color':'blue','size':20}
font2 = {'family':'serif','color':'darkred','size':15}
plt.plot(x,y)
plt.xlabel("Overs',fontdict=font2)
plt.ylabel("Runs' ,fontdict=font2)
Plt.title("Sport Data" ,fontdict=font1)
plt.show()
```

Set Font Properties for Title and Labels

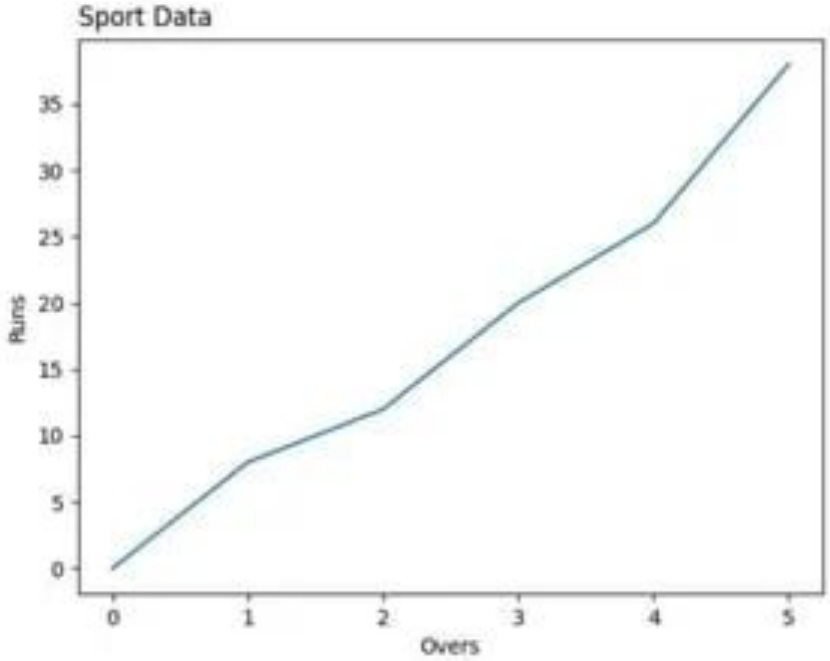


Position the Title

You can use the loc parameter in title() to position the title.

Legal values are: 'left', 'right', and 'center'. Default value is 'center'.

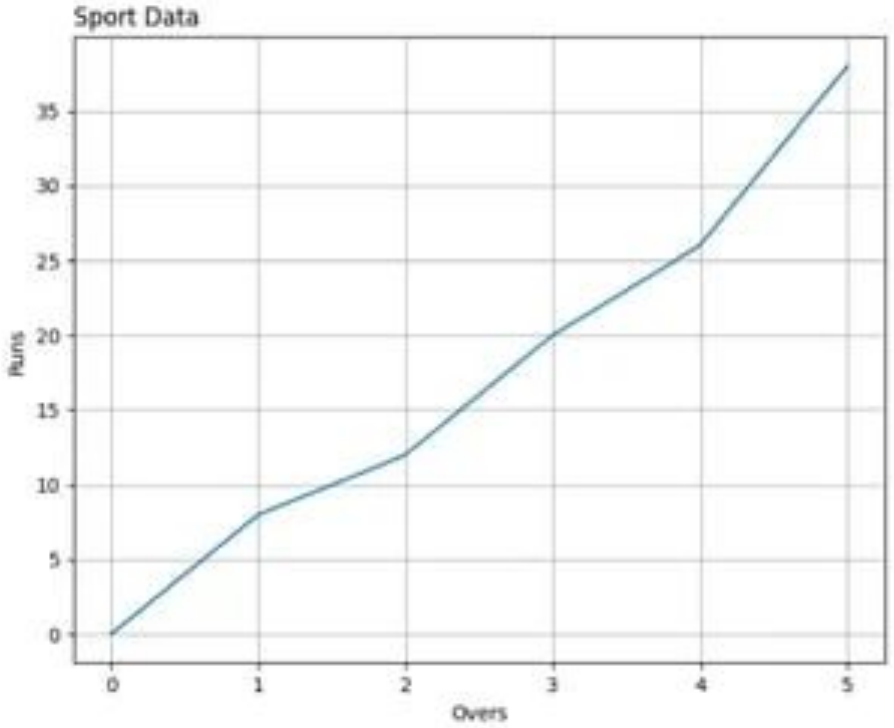
```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([0,1,2,3,4,5])
y = np.array([0,8,12,20,26,38])
plt.plot(x,y)
plt.xlabel("Overs')
plt.ylabel("Runs')
Plt.title("Sport Data", loc='left')
plt.show()
```



Add Grid Lines to a Plot

With Pyplot, you can use the grid() function to add grid lines to the plot.

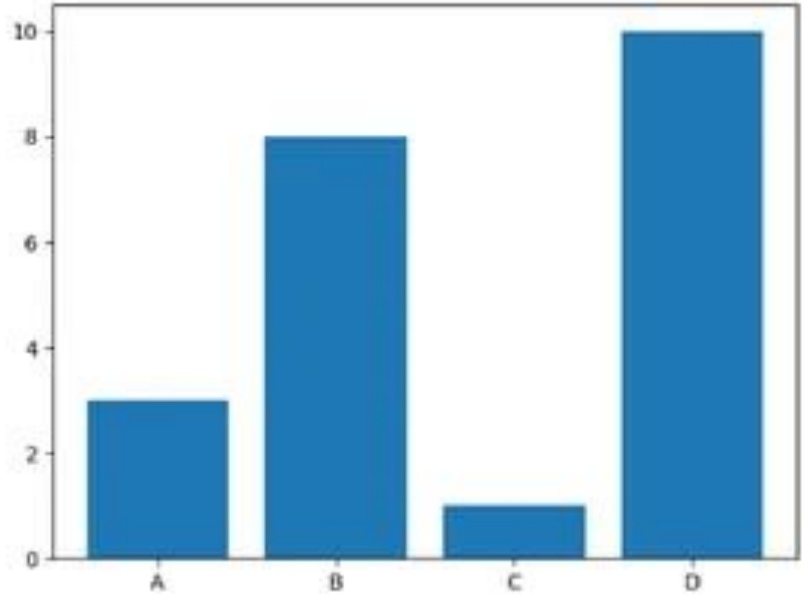
```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([0,1,2,3,4,5])
y = np.array([0,8,12,20,26,38])
plt.plot(x,y)
plt.xlabel("Overs')
plt.ylabel("Runs')
Plt.title("Sport Data", loc='left')
Plt.grid()
plt.show()
```



Create Bar

With Pyplot, you can use the bar() function to draw bar graphs:

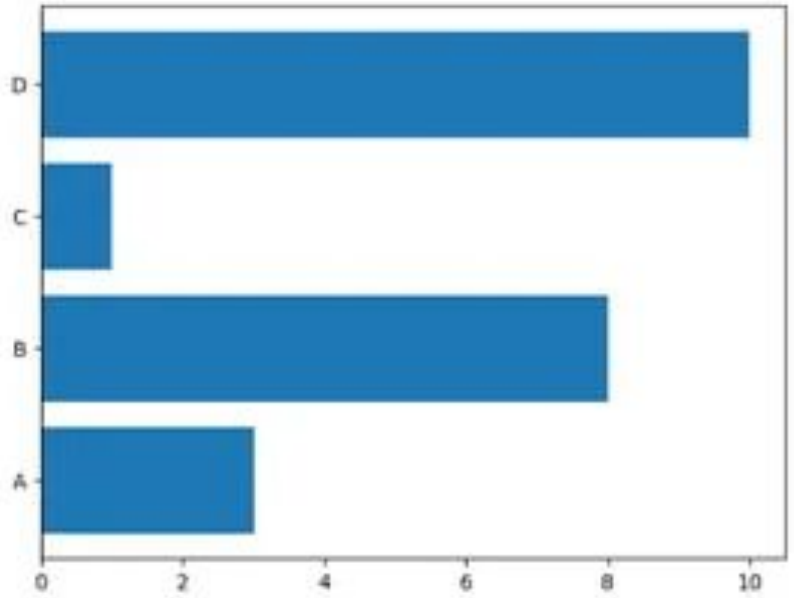
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(['A','B','C','D'])
y = np.array([3,8,1,10])
plt.bar(x,y)
plt.show()
```



Create Horizontal Bar

If you want the bars to be displayed horizontally instead of vertically, use the barh() function:

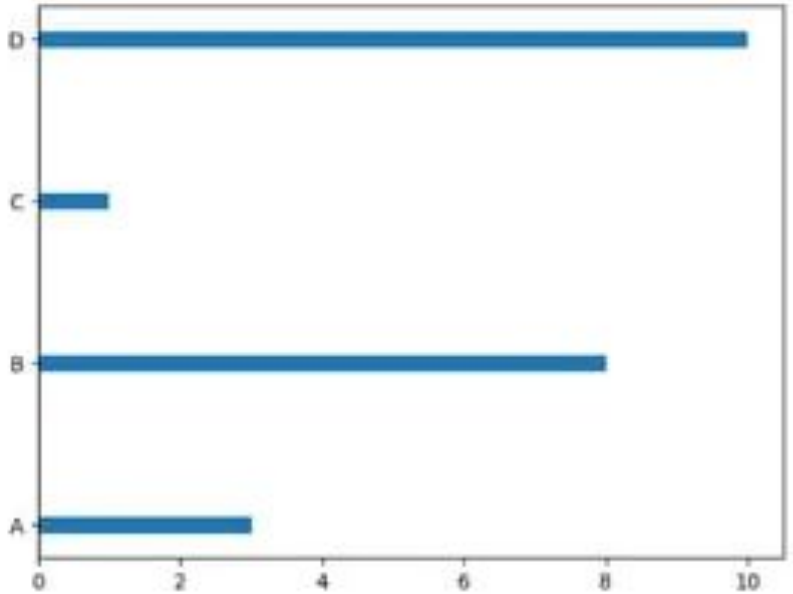
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(['A','B','C','D'])
y = np.array([3,8,1,10])
plt.barh(x,y)
plt.show()
```



Bar Width

The bar() takes the keyword argument width to set the width of the bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(['A','B','C','D'])
y = np.array([3,8,1,10])
plt.barh(x,y,width=0.1)
plt.show()
```



Creating Pie Charts

With Pyplot, you can use the pie() function to draw pie charts:

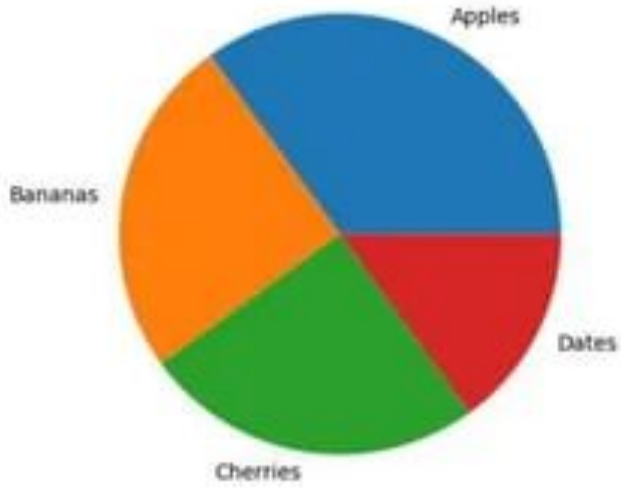
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([35,25,25,15])

plt.pie(y)
plt.show()
```

Labels

Add labels to the pie chart with the label parameter.

The label parameter must be an array with one label for each wedge:

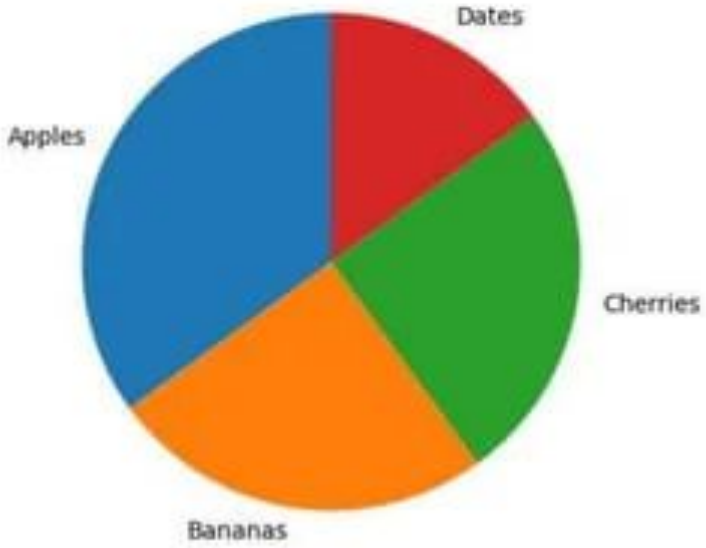
```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([35,25,25,15])
mylabels = ["Apples", "Bananas",
"Cherries", "Dates"]
plt.pie(y, labels=mylabels)
plt.show()
```



Start Angle

As mentioned the default start angle is at the x-axis, but you can change the start angle by specifying a startangle parameter.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([35,25,25,15])
mylabels = ["Apples", "Bananas",
"Cherries", "Dates"]
plt.pie(y, labels=mylabels,startangle=90)
plt.show()
```

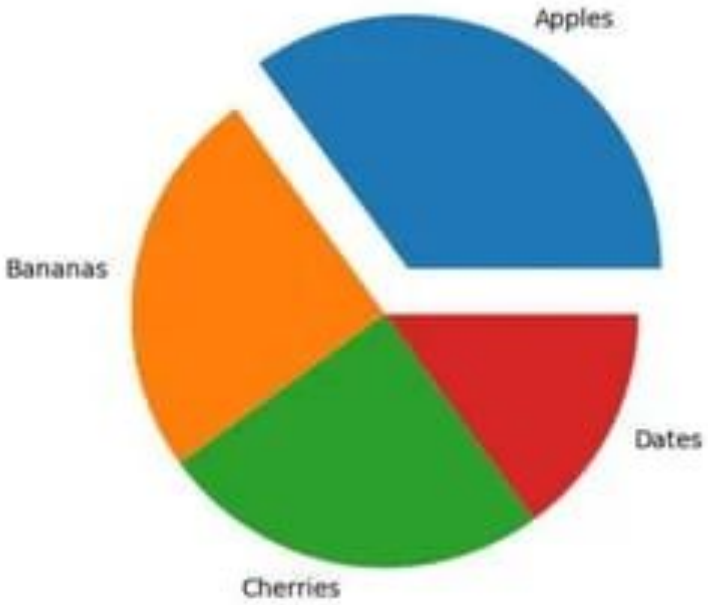


Explode

Maybe you want one of the wedges to stand out? The explode parameter allows you to do that.

The explode parameter, if specified, and not None, must be an array with one value for each wedge

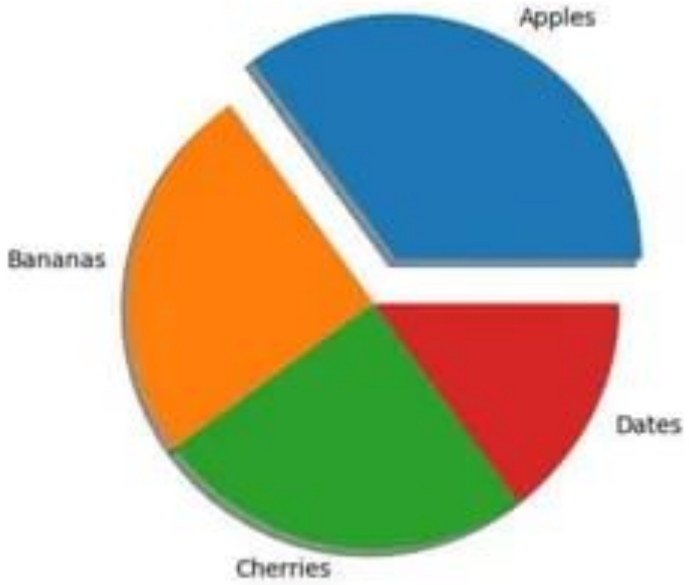
```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([35,25,25,15])
mylabels = ["Apples", "Bananas", "Cherries",
"Dates"]
myexplode = [0.2, 0, 0, 0]
plt.pie(y, labels=mylabels,
startangle=90,explode=myexplode)
plt.show()
```



Shadow

Add a shadow to the pie chart by setting the shadows parameter to True:

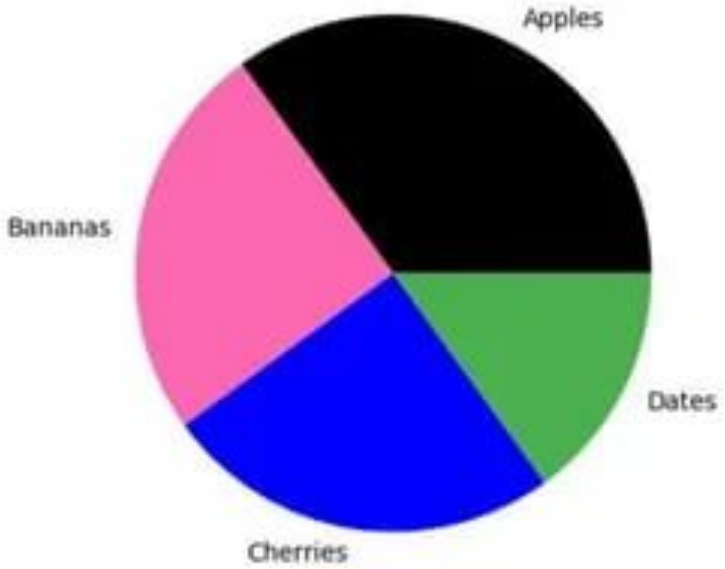
```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([35,25,25,15])
mylabels = ["Apples", "Bananas", "Cherries",
"Dates"]
myexplode = [0.2, 0, 0, 0]
plt.pie(y, labels=mylabels, startangle=90,
explode=myexplode, shadow=True)
plt.show()
```



Colors

You can set the color of each wedge with the colors parameter.

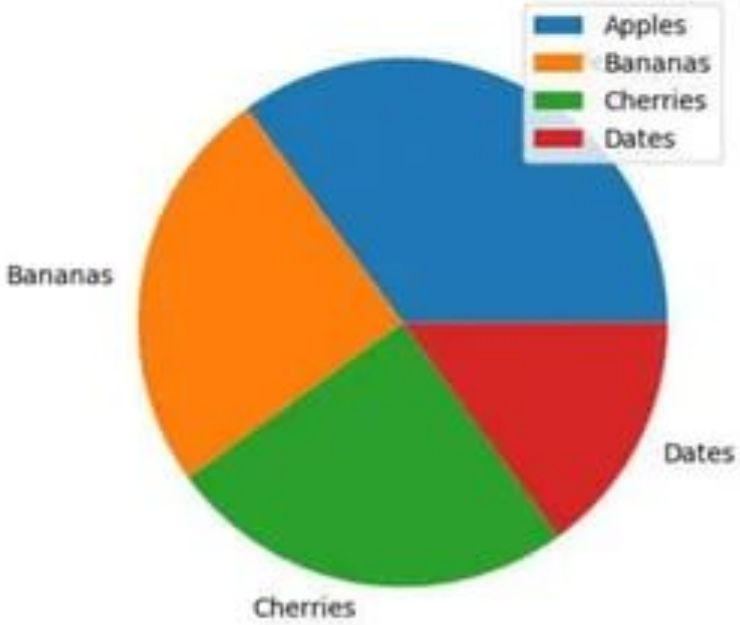
```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([35,25,25,15])
mylabels = ["Apples", "Bananas", "Cherries",
"Dates"]
mycolors = ["black", "hotpink", "b", "#4CAF50"]
myexplode = [0.2, 0, 0, 0]
plt.pie(y, labels=mylabels, startangle=90,
explode=myexplode, colors= mycolors)
plt.show()
```



Legend

To add a list of explanation for each wedge, use the legend() function:

```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([35,25,25,15])
mylabels = ["Apples", "Bananas", "Cherries",
"Dates"]
plt.pie(y, labels=mylabels)
plt.legend()
plt.show()
```



Legend With Header

To add a header to the legend, add the title parameter to the legend function.

```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([35,25,25,15])
mylabels = ["Apples", "Bananas", "Cherries",
"Dates"]
plt.pie(y, labels=mylabels)
plt.legend(title='Four Fruits:')
plt.show()
```